8-5 · 데이터 파이프라인·최종 RAG 챗봇 · 실습 1H

엔드투엔드 검증·시연

새 문서 하나가 답변에 나오는가 — 전 구간의 증명

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 대조군 → 새 문서 투입 → DAG 실행 → 답변 반영을 처음부터 끝까지 성공시킨다

② 두 번 돌려 count 가 12 그대로인 것(멱등)을 확인한다

③ Qdrant 를 내렸다 올려 retries=2 가 스스로 회복하는 것을 본다

④ submit/ 에 일곱 파일을 모으고 키가 섞이지 않았는지 grep 한다

재기 전의 값을 남기는 것이 검증의 기본 — count 11 부터 적어 두세요

실습 흐름

여덟 단계 — 한 줄 시나리오를 처음부터 끝까지

실무라면 투입구는 S3 버킷이나 업로드 화면 — 오늘은 dags/data/incoming/ 입니다

1·2. 기준선	3. 투입	4·5. 실행·확인	6·7. 멱등·회복	8. 산출물
count 11 0.253 < 0.4	incoming/ lakehouse.txt	unpause · trigger 12 · 0.517	두 번 돌려 12 docker stop qdrant	submit/ 일곱 파일 grep "sk-"

1~3단계

기준선, 대조군, 그리고 투입

1. **.../collections/lecture_kb/points/count → 11 을 적어 둔다**
   이 숫자와 비교해야 「늘었다」를 말할 수 있다

2. **챗봇에 「레이크하우스가 뭐야?」**
   「찾지 못했습니다 (0.253 < 기준 0.4)」(실측) — 지식베이스에 없는 말

3. **cat > dags/data/incoming/lakehouse.txt << 'EOF'**
   두 줄 — 레이크의 저장 + 웨어하우스의 관리, Iceberg 로 트랜잭션·스키마 변경

4. **ls dags/data/incoming/**
   airflow_ops.txt · lakehouse.txt 두 파일

대조군

0.253

< 0.4

4단계

최종 DAG 를 돌리세요 — 11 에서 12 로

8-3 DAG 끄기	af dags pause rag_pipeline — 같은 lecture_kb 를 두 DAG 가 매일 손보지 않게
켜기	af dags unpause rag_pipeline_final
실행	af dags trigger rag_pipeline_final → sleep 60
실행 id	RUN=$(af dags list-runs rag_pipeline_final | grep -o "manual__[^]*" | head -1)
상태	af tasks states-for-dag-run rag_pipeline_final "$RUN" -o plain
세기	.../collections/lecture_kb/points/count → 12

다섯 태스크 success · count 12 — unpause 는 매일 자동 실행도 켠다: 다음 자정(UTC)부터 사람이 손대지 않아도 돈다

5단계

같은 질문을 다시 — 전과 후

전 — 2단계

0.253 < 0.4 → 「찾지 못했습니다」

- 지식베이스 11 조각(7주차 셋 + airflow_ops.txt)
- 「레이크하우스」는 문서에 없다
- 근거의 적재 시각 「알 수 없음」

VS

후 — 5단계

유사도 0.517 · lakehouse.txt 가 1등(실측)

- minio://raw-docs/incoming/lakehouse.txt 를 근거로 답한다
- 「검색 근거 보기」에 적재 시각이 찍힌다
- 사이드바 조각 수 12 (새로고침)

5단계

이 한 번의 문답이 전 구간이 살아 있다는 증명

수집	Lake	검증	임베딩·적재	챗봇
incoming/ *.txt	raw-docs/ incoming/	20자 미만 막는다	build_kb + ingested_at	근거 + 적재 시각

근거 보기를 펼치면 방금 넣은 그 문장과 적재 시각이 보인다

어느 한 구간이라도 끊기면 실패한다 — 그래서 단계별 확인보다 강하다

6단계

두 번 돌려 세어 보세요 — 멱등의 증거

1. **af dags trigger rag_pipeline_final → sleep 60**
   같은 투입구, 같은 문서 둘

2. **.../points/count → 12 그대로**
   두 번째 실행도 문서마다 지우고 다시 넣었기 때문

3. **바뀌는 것은 적재 시각뿐**
   챗봇 근거 보기에서 이번 실행 시각으로 바뀐 것을 확인

실측

12 →

12

7단계 — 일부러 실패시키기

Qdrant 를 내리고 돌려 재시도를 눈으로

retries=2 · retry_delay 1분이 실제로 무엇을 해 주는지 봅니다 — 다시 켜도 네트워크 연결은 그대로

1. 정지	2. 트리거	3. 30초 뒤 상태	4. 1분 안에 복구	5. 다시 상태
docker stop qdrant	af dags trigger rag_pipeline_final	build_index up_for_retry	docker start qdrant sleep 120	다섯 줄 모두 success

8단계

submit/ 일곱 파일과 마지막 grep

파일	어떻게
app.py · rag_env.py	cp — ~/data-eng/week8
rag_pipeline_final.py build_kb.py · minio_env.py	cp — ~/airflow-lab/dags
requirements.txt	pip freeze > …
.env.example	printf 로 키 이름만
.env(실제 키)	제출물 · 저장소에 넣지 않는다
마지막 확인	grep -rn "sk-" submit/ → 비어 있어야

ls -a submit 에 일곱 파일 · grep 결과가 비어 있으면 제출 준비 완료 — README 는 아키텍처 · 실행 순서 · 내 숫자(11 → 12 · 0.253 → 0.517)

확인 포인트

이 1시간이 끝날 때 확인할 것

1. count 11 → 12 (lakehouse.txt 한 조각) · 다섯 태스크 success
2. 「레이크하우스가 뭐야?」 — 전 0.253 찾지 못함, 후 0.517 lakehouse.txt
3. 두 번째 trigger 뒤에도 12 — 적재 시각만 바뀐다
4. Qdrant 를 내린 동안 build_index up_for_retry → 올리면 다섯 줄 success
5. ls -a submit 에 일곱 파일 · grep "sk-" 결과가 비어 있다

팀별 시연 — 「대조군 질문 → 투입 → 트리거 → 같은 질문에 답」을 처음부터 끝까지 한 번에

8-5 · 실습 1H 마무리

다음은 과정 회고·산출물 제출 확인 (0.5H)

8주 동안 만든 것을 돌아보고, 주차별 산출물이 제출 형태로 갖춰졌는지 확인합니다

- ✓ 무엇을 만들었나 — 플랫폼·파이프라인·챗봇
- ✓ 주차별 산출물 제출 확인 표
- ✓ 자주 겪는 오류 네 가지
- ✓ 다음에 공부할 것과 심화 과제

가져갈 것

시연 가능한 submit/ 폴더(키 없음)